

LAB GUIDE WITH SOLUTIONS

Redis Lab Guide

Ten hands-on labs, with solutions inline

Module 07 · NoSQL & Redis

ICICI Bank Learning Academy

Enterprise Data Engineering with SQL, Redis & Kafka

Module 07 · NoSQL & Redis · Instructor-led · For classroom use

How to use this lab guide

Ten labs, sized for a six-hour hands-on session. Every lab states its objective, lists the steps, shows the exact commands and the output you should see, and ends with a checkpoint you can verify before moving on. Solutions are inline - this document is the instructor copy and the participant copy at the same time.

All commands are written for **redis-cli** on Windows via WSL2. They work unchanged on Linux, in Docker (prefix with `docker exec -it icici-redis`) and against Memurai (use `memurai-cli`).

Lab	Title	Covers	Time
7.1	Install Redis and meet redis-cli	Deck 2	30 min
7.2	Key commands: lifecycle, expiry, discovery	Deck 5	35 min
7.3	Strings, counters and a rate limiter	Deck 5	35 min
7.4	Hashes: the customer profile cache	Decks 3, 5	30 min
7.5	Lists and Sets: a queue and a segment	Decks 3, 5	40 min
7.6	Sorted Sets: leaderboard and delayed jobs	Decks 3, 5	35 min
7.7	Look inside: encodings, memory, RESP	Decks 3, 4	30 min
7.8	redis.conf, persistence and eviction	Deck 4	35 min
7.9	Cache-aside end to end, in Node.js	Decks 2, 5	45 min
7.10	Transactions, Lua and a distributed lock	Deck 5	35 min

WATCH OUT Keys created in these labs all start with `icici:` so you can find and remove them afterwards with a single `SCAN` loop. Never run `FLUSHALL` on a shared server.

READ THIS Conventions: lines starting with `$` are shell commands. Lines with no prefix inside a `redis-cli` block are commands you type at the `127.0.0.1:6379>` prompt. Indented lines in teal are the output you should see.

LAB 7.1 · 30 MINUTES

Install Redis and meet redis-cli

Objective	Prerequisite
Get a Redis 7.x server running on your Windows machine and prove it works.	Windows 10 2004+ or Windows 11, administrator rights.

Step 1 · Install WSL2 and Ubuntu

```
# PowerShell, as Administrator
wsl --install -d Ubuntu
# reboot when prompted, then set a UNIX username and password

wsl -l -v
  NAME      STATE      VERSION
  Ubuntu    Running    2
```

TIP If WSL is blocked by policy, use Docker instead: `docker run -d --name icici-redis -p 6379:6379 redis:7-alpine redis-server --appendonly yes` and then `docker exec -it icici-redis redis-cli`. Everything after this step is identical.

Step 2 · Install and start the server

```
# inside the Ubuntu shell
sudo apt update
sudo apt install -y redis-server
sudo service redis-server start
sudo service redis-server status
  * redis-server is running
```

Step 3 · Verify

```
redis-cli ping
  PONG
redis-cli INFO server | head -4
  redis_version:7.2.4
  os:Linux 5.15.167.4-microsoft-standard-WSL2 x86_64
redis-cli DBSIZE
  (integer) 0
```

Step 4 · Your first keys

```
$ redis-cli
127.0.0.1:6379> SET icici:greeting "Hello from ICICI Bank"
  OK
127.0.0.1:6379> GET icici:greeting
  "Hello from ICICI Bank"
127.0.0.1:6379> TYPE icici:greeting
  string
127.0.0.1:6379> EXISTS icici:greeting
  (integer) 1
127.0.0.1:6379> DEL icici:greeting
```

```
(integer) 1
127.0.0.1:6379> EXISTS icici:greeting
(integer) 0
```

Step 5 · Expiry, seen for the first time

```
SET icici:bal:9003 341244.80 EX 20
OK
TTL icici:bal:9003
(integer) 18
GET icici:bal:9003
"341244.80"
# wait 20 seconds
GET icici:bal:9003
(nil)
TTL icici:bal:9003
(integer) -2      # -2 = no such key, -1 = no TTL
```

CHECKPOINT PING returns PONG, a key survives, and a key with a TTL disappears on its own. If all three are true, your environment is ready.

LAB 7.2 · 35 MINUTES

Key commands: lifecycle, expiry and discovery

Objective	Prerequisite
Learn every key-level command a developer needs, and why SCAN replaces KEYS.	Lab 7.1 complete.

Step 1 · Create a small keyspace

```
MSET icici:bal:9003 341244.80 icici:bal:7711 89210.50 \
icici:bal:4502 12500.00 icici:bal:8890 764300.25
OK
HSET icici:cust:9003 name "A. Sharma" tier gold
(integer) 2
RPUSH icici:recent:9003 txn:881 txn:882 txn:883
(integer) 3
DBSIZE
(integer) 6
```

Step 2 · Inspect keys without knowing their type

```
TYPE icici:bal:9003
string
TYPE icici:cust:9003
hash
TYPE icici:recent:9003
list
EXISTS icici:bal:9003 icici:bal:7711 icici:bal:0000
(integer) 2          # counts how many exist, not a boolean
```

Step 3 · Rename, copy and delete

```
COPY icici:bal:9003 icici:bal:9003:backup
(integer) 1
RENAME icici:bal:9003:backup icici:bal:9003:snap
OK
RENAMENX icici:bal:9003:snap icici:bal:7711
(integer) 0          # refused - target already exists
UNLINK icici:bal:9003:snap
(integer) 1          # non-blocking delete, prefer over DEL
```

Step 4 · Expiry in all its forms

```
EXPIRE icici:bal:7711 300
(integer) 1
TTL icici:bal:7711
(integer) 298
EXPIRE icici:bal:7711 120 GT
(integer) 0          # GT refuses to SHORTEN an existing TTL
EXPIRE icici:bal:7711 600 GT
(integer) 1          # extending is allowed
PERSIST icici:bal:7711
```

```
(integer) 1
TTL icici:bal:7711
(integer) -1      # -1 means no TTL
```

Step 5 · The TTL trap

This is the single most common Redis bug in production code. Run it and watch the TTL vanish.

```
SET icici:sess:8f3a "{}" EX 1800
OK
TTL icici:sess:8f3a
(integer) 1798
SET icici:sess:8f3a "{updated}"      # no EX!
OK
TTL icici:sess:8f3a
(integer) -1      # the session is now immortal

# the fix
SET icici:sess:8f3a "{}" EX 1800
SET icici:sess:8f3a "{updated}" KEPTTL
TTL icici:sess:8f3a
(integer) 1797    # preserved
```

Step 6 · KEYS versus SCAN

```
# what NOT to do - O(n) over the whole keyspace, blocks everyone
KEYS icici:bal:*
1) "icici:bal:7711"  2) "icici:bal:4502" ...

# what to do - cursor based, never blocks
SCAN 0 MATCH icici:bal:* COUNT 10
1) "0"              <- cursor 0 means the scan is complete
2) 1) "icici:bal:7711" 2) "icici:bal:4502" ...

# from the shell, the loop is done for you
$ redis-cli --scan --pattern 'icici:bal:*
```

WATCH OUT SCAN gives a weak guarantee: every key present for the whole scan is returned at least once, and a key may be returned twice. Your code must tolerate duplicates. That is the price of not blocking the server.

Solution · Clean up only this lab's keys, safely

```
$ redis-cli --scan --pattern 'icici:*' | xargs -r redis-cli UNLINK
$ redis-cli DBSIZE
(integer) 0
```

CHECKPOINT You can name, inspect, expire and find keys without ever running KEYS or FLUSHALL.

LAB 7.3 · 35 MINUTES

Strings, counters and a rate limiter

Objective	Prerequisite
Use the SET options properly, then build a working per-client API rate limiter.	Lab 7.2 complete.

Step 1 · The SET options that matter

```
SET icici:bal:9003 341244.80 EX 300
OK                                     # set with a 5 minute TTL
SET icici:bal:9003 999 NX
(nil)                                # NX refused - the key exists
SET icici:bal:0001 100 NX
OK                                   # NX succeeded - key was absent
SET icici:bal:9003 350000 XX GET
"341244.80"                           # XX required it to exist; GET returned the old value
GETEX icici:bal:9003 EX 600
"350000"                             # read and re-arm the TTL in one round trip
GETDEL icici:bal:0001
"100"                               # read it and remove it atomically
```

Step 2 · Batch reads and writes

```
MSET icici:rate:USDINR 83.42 icici:rate:EURINR 90.11 \
icici:rate:GBPINR 105.77
OK
MGET icici:rate:USDINR icici:rate:EURINR icici:rate:GBPINR
1) "83.42" 2) "90.11" 3) "105.77"
```

One MGET is one network round trip. Three GETs are three. On a 0.4 ms link that is 0.4 ms against 1.2 ms for identical work - and the gap grows linearly with the number of keys.

Step 3 · Atomic counters

```
INCR icici:cnt:txn:2026-09-21
(integer) 1
INCRBY icici:cnt:txn:2026-09-21 49
(integer) 50
INCRBYFLOAT icici:sum:txn:2026-09-21 2500.75
"2500.75"
SET icici:cnt:bad "abc"
INCR icici:cnt:bad
(error) ERR value is not an integer or out of range
```

INCR is atomic on the server. A hundred application instances can increment the same counter with no locks, no lost updates and no read-modify-write cycle.

Step 4 · Build the rate limiter

Requirement: each client may call the balance API 5 times per minute. Reject the sixth.

```
# the key carries the minute, so it cleans itself up
INCR icici:rl:9003:202609211430
```

```
(integer) 1
EXPIRE icici:rl:9003:202609211430 60 NX
(integer) 1          # NX: only set the TTL on the first call

# calls 2 to 5
INCR icici:rl:9003:202609211430
(integer) 2 ... 5

# call 6 - the application sees 6 > 5 and returns HTTP 429
INCR icici:rl:9003:202609211430
(integer) 6
TTL icici:rl:9003:202609211430
(integer) 41          # resets itself when the minute ends
```

Solution · Why EXPIRE ... NX and not a plain EXPIRE

A plain EXPIRE on every call re-arms the 60-second window each time, so a client calling steadily would never reset. NX sets the TTL only when the key was just created, which pins the window to the first call of that minute. Lab 7.10 makes the INCR and the EXPIRE a single atomic step with Lua, which closes the remaining crash window.

CHECKPOINT You can explain what NX, XX, EX, KEeptTL and GET each do, and you have a rate limiter that resets on its own with no cleanup job.

LAB 7.4 · 30 MINUTES

Hashes: the customer profile cache

Objective	Prerequisite
Cache a record as a Hash and update single fields without rewriting the whole object.	Lab 7.3 complete.

Step 1 · Write and read a record

```
HSET icici:cust:9003 name "A. Sharma" tier gold \
    branch BR-MUM-014 kyc verified since 2016-04-11
(integer) 5
HGET icici:cust:9003 tier
"gold"
HMGET icici:cust:9003 name branch
1) "A. Sharma" 2) "BR-MUM-014"
HGETALL icici:cust:9003
1) "name" 2) "A. Sharma" 3) "tier" 4) "gold" ...
HLEN icici:cust:9003
(integer) 5
```

Step 2 · Update one field only

```
HSET icici:cust:9003 tier platinum
(integer) 0 # 0 = field existed and was updated
HINCRBY icici:cust:9003 logins 1
(integer) 1 # the field is created at 0 first
HEXISTS icici:cust:9003 pan
(integer) 0
HDEL icici:cust:9003 since
(integer) 1
```

Step 3 · One TTL for the whole record

```
EXPIRE icici:cust:9003 900
(integer) 1
TTL icici:cust:9003
(integer) 897
HSET icici:cust:9003 tier gold
TTL icici:cust:9003
(integer) 894 # writing a field does NOT reset the TTL
```

WATCH OUT Note the difference from strings: HSET does not clear the key's TTL, but a plain SET does. That asymmetry catches people out - it is a good reason to prefer Hashes for cached records.

Step 4 · Hash versus JSON string - the comparison that decides it

Requirement	Hash	JSON in a String
Read one field	HGET - one field over the wire	GET the whole blob, parse it in the app
Update one field	HSET - atomic, server-side	GET, parse, modify, serialise, SET
Two servers update different fields	Both succeed	Last writer wins - the other update is lost

Requirement	Hash	JSON in a String
Increment a counter field	HINCRBY - atomic	Read-modify-write race
Memory for a small record	listpack, very compact	One string, also compact
Nested structure	Not supported - flatten it	Natural

Solution · The rule

Use a Hash when the fields are read or written independently. Use a JSON string when the record is always read and written as a whole and has nested structure. For a customer profile cache that the app updates field by field, the Hash wins on both correctness and bandwidth.

CHECKPOINT You can store a record, read and write single fields, and explain why the lost-update problem does not arise with a Hash.

LAB 7.5 · 40 MINUTES

Lists and Sets: a queue and a segment

Objective	Prerequisite
Build a working job queue with a blocking worker, then compute a cross-sell segment.	Lab 7.4 complete.

Part A · A statement-generation job queue

Step 1 · Producer

```
LPUSH icici:q:statements '{"acct":9003,"month":"2026-08"}'
(integer) 1
LPUSH icici:q:statements '{"acct":7711,"month":"2026-08"}'
(integer) 2
LLEN icici:q:statements
(integer) 2
LRANGE icici:q:statements 0 -1
1) "{\"acct\":\"7711...}\" 2) "{\"acct\":\"9003...\"}
```

Step 2 · Worker - open a SECOND redis-cli window for this

```
# blocks for up to 30 seconds waiting for work
BRPOP icici:q:statements 30
1) "icici:q:statements"
2) "{\"acct\":\"9003,\"month\":\"2026-08\"}"

# run it again with an empty queue
BRPOP icici:q:statements 5
(nil) # timed out after 5 seconds
```

LPUSH on the left and BRPOP on the right makes a FIFO queue: the oldest job is taken first. BRPOP blocks inside the server, so the worker does no polling and burns no CPU while idle.

Step 3 · The bounded log - last 10 transactions

```
LPUSH icici:recent:9003 txn:881 txn:882 txn:883 txn:884
(integer) 4
LTRIM icici:recent:9003 0 9
OK # keep only the newest 10, forever
LRANGE icici:recent:9003 0 2
1) "txn:884" 2) "txn:883" 3) "txn:882"
```

TIP LPUSH followed by LTRIM is the standard bounded-log idiom. The list can never grow without limit, so this key has a predictable memory cost no matter how long the application runs.

Part B · Customer segments with Sets

Step 4 · Build the segments

```
SADD icici:seg:salaried 9003 7711 4502 8890
(integer) 4
SADD icici:seg:salaried 9003
(integer) 0 # already a member - no duplicate
SADD icici:seg:creditcard 7711 8890 1120
```

```
(integer) 3
SCARD icici:seg:salaried
(integer) 4
SISMEMBER icici:seg:salaried 9003
(integer) 1
```

Step 5 · Set algebra, computed in the server

```
# who is salaried AND has a credit card
SINTER icici:seg:salaried icici:seg:creditcard
1) "7711" 2) "8890"

# the cross-sell target list: salaried, NO credit card
SDIFF icici:seg:salaried icici:seg:creditcard
1) "9003" 2) "4502"

# everyone in either segment
SUNION icici:seg:salaried icici:seg:creditcard
1) "9003" 2) "7711" 3) "4502" 4) "8890" 5) "1120"

# just the size, without building the result
SINTERCARD 2 icici:seg:salaried icici:seg:creditcard
(integer) 2
```

Step 6 · Idempotency with a Set

```
SADD icici:seen:req REQ-88213
(integer) 1 # new - process the payment
SADD icici:seen:req REQ-88213
(integer) 0 # duplicate - reject, do not process twice
```

Solution · Why SSCAN, not SMEMBERS

A real segment has millions of members. SMEMBERS builds the whole reply in memory and blocks the single thread while it does. SSCAN walks it in batches:

```
SSCAN icici:seg:salaried 0 COUNT 100
1) "0"
2) 1) "9003" 2) "7711" 3) "4502" 4) "8890"
```

CHECKPOINT You have a queue whose worker never polls, a log that cannot grow without limit, and a segment intersection computed without moving any data to the application.

LAB 7.6 · 35 MINUTES

Sorted Sets: leaderboard and delayed jobs

Objective	Prerequisite
Use one type for two problems that look unrelated: live ranking, and scheduling.	Lab 7.5 complete.

Part A · A live branch leaderboard

```

ZADD icici:lb:branch:2026-09 1842300 BR-MUM-014 \
    1611050 BR-DEL-007 1402880 BR-BLR-022 \
    1190400 BR-CHN-003
(integer) 4

# a transaction posts - update the score atomically
ZINCRBY icici:lb:branch:2026-09 25000 BR-CHN-003
"1215400"

# top 3, highest first
ZREVRANGE icici:lb:branch:2026-09 0 2 WITHSCORES
1) "BR-MUM-014" 2) "1842300"
3) "BR-DEL-007" 4) "1611050"
5) "BR-BLR-022" 6) "1402880"

# where does one branch stand?
ZREVRANK icici:lb:branch:2026-09 BR-CHN-003
(integer) 3 # 0-based: 4th place
ZSCORE icici:lb:branch:2026-09 BR-CHN-003
"1215400"
ZCARD icici:lb:branch:2026-09
(integer) 4

```

No sort step, no batch job, no recalculation. The ranking is correct the instant ZINCRBY returns, and reading the top 3 costs $O(\log n + 3)$.

Part B · A delayed job scheduler

The trick: use the due timestamp as the score. The sorted set is then a priority queue.

```

# schedule three jobs, score = unix time they are due
ZADD icici:jobs:due 1758412800 "job:eod-recon" \
    1758416400 "job:sms-batch" \
    1758499200 "job:monthly-stmt"
(integer) 3

# the scheduler loop: everything due up to now
ZRANGEBYSCORE icici:jobs:due 0 1758416400 LIMIT 0 10
1) "job:eod-recon" 2) "job:sms-batch"

# after dispatching, drop that window
ZREMRANGEBYSCORE icici:jobs:due 0 1758416400
(integer) 2
ZCARD icici:jobs:due
(integer) 1

```

```
# or pop the single most urgent job
ZPOPMIN icici:jobs:due
  1) "job:monthly-stmt"  2) "1758499200"
```

Step 3 · GT and LT - only move a score one way

```
ZADD icici:lb:branch:2026-09 GT 1000000 BR-MUM-014
(integer) 0          # refused: 1000000 < current 1842300
ZADD icici:lb:branch:2026-09 GT 1900000 BR-MUM-014
(integer) 0          # updated (0 = no NEW member added)
ZSCORE icici:lb:branch:2026-09 BR-MUM-014
"1900000"
```

TIP GT is how you record a high-water mark - a peak balance, a best response time - without reading the current value first. The check happens inside the server, so there is no race.

Solution · Why not a List with a sort in the application?

A list would need the application to fetch every element, sort it, and write it back on every single update - $O(n)$ work per transaction, and a lost-update race between any two application servers. The sorted set does the same job in $O(\log n)$ with no race, because the ordering is maintained inside the single-threaded server.

CHECKPOINT You can maintain a live ranking and a due-time scheduler with the same type, and you know when to reach for GT.

LAB 7.7 · 30 MINUTES

Look inside: encodings, memory and RESP

Objective	Prerequisite
Watch Redis switch encodings, measure real memory, and read the wire protocol.	Lab 7.6 complete.

Step 1 · Watch an encoding change

```
DEL icici:demo:list
RPUSH icici:demo:list a b c d e
(integer) 5
OBJECT ENCODING icici:demo:list
"listpack"           # compact, flat, cache-friendly

# push past list-max-listpack-size (default 128)
$ for i in $(seq 1 200); do redis-cli RPUSH icici:demo:list "v$i" > /dev/null; done

OBJECT ENCODING icici:demo:list
"quicklist"          # converted - and it never converts back
```

Step 2 · The same for Sets and Hashes

```
SADD icici:demo:ints 1 2 3 4 5
OBJECT ENCODING icici:demo:ints
"intset"             # all members are integers
SADD icici:demo:ints "nine-thousand-three"
OBJECT ENCODING icici:demo:ints
"listpack"           # one string member and it converts

SET icici:demo:s "short value"
OBJECT ENCODING icici:demo:s
"embstr"              # 44 bytes or fewer
SET icici:demo:n 12345
OBJECT ENCODING icici:demo:n
"int"                 # stored as a native integer
```

Step 3 · Measure real memory

```
MEMORY USAGE icici:cust:9003
(integer) 312          # bytes, including key and overhead
INFO memory | head -6
used_memory_human:1.23M
used_memory_rss_human:9.14M
mem_fragmentation_ratio:7.43

$ redis-cli --bigkeys
[00.00%] Biggest list found so far 'icici:demo:list' with 205 items
$ redis-cli --memkeys
```

Compare the cost of one Hash of 5 fields against 5 separate String keys holding the same data. The per-key overhead - roughly 50 to 90 bytes for the key name, the dict entry and the object header - is what makes fewer, larger keys cheaper.

Step 4 · Read RESP off the wire

```
# talk to Redis with netcat, no client library
$ (printf 'PING\r\n'; sleep 1) | nc 127.0.0.1 6379
+PONG

$ (printf '*3\r\n$3\r\nSET\r\n$4\r\ndemo\r\n$2\r\nhi\r\n'; sleep 1) \
| nc 127.0.0.1 6379
+OK

$ (printf '*2\r\n$3\r\nGET\r\n$4\r\ndemo\r\n'; sleep 1) | nc 127.0.0.1 6379
$2
hi
```

First byte	Type	Example
+	Simple string	+OK
-	Error	-ERR unknown command
:	Integer	:42
\$	Bulk string (length-prefixed)	\$2 then hi
*	Array (count, then elements)	*2 then two bulk strings

Step 5 · Find the slow commands

```
CONFIG SET slowlog-log-slower-than 0      # log everything, briefly
KEYS *
CONFIG SET slowlog-log-slower-than 10000  # back to 10 ms
SLOWLOG GET 3
  1) 1) (integer) 14          # entry id
     2) (integer) 1758412812
     3) (integer) 1832        # microseconds
     4) 1) "KEYS"  2) "*"
SLOWLOG RESET
```

CHECKPOINT You can predict an encoding from the data, measure what a key really costs, read a RESP exchange by eye, and find the slowest command the server has seen.

LAB 7.8 · 35 MINUTES

redis.conf, persistence and eviction

Objective	Prerequisite
Configure Redis the way production runs it, and watch eviction and persistence work.	Lab 7.7 complete. Do this on your own instance, never a shared one.

Step 1 · Read the running configuration

```
CONFIG GET maxmemory
  1) "maxmemory"  2) "0"          # 0 = no limit, the default
CONFIG GET maxmemory-policy
  1) "maxmemory-policy"  2) "noeviction"
CONFIG GET appendonly
  1) "appendonly"  2) "no"
CONFIG GET save
  1) "save"  2) "3600 1 300 100 60 10000"
CONFIG GET dir
  1) "dir"  2) "/var/lib/redis"
```

Step 2 · Make it behave like a cache and watch eviction

```
CONFIG SET maxmemory 3mb
OK
CONFIG SET maxmemory-policy allkeys-lru
OK

# fill it past the limit
$ redis-cli DEBUG POPULATE 200000 icici:fill:

INFO stats | grep evicted_keys
  evicted_keys:41208          # Redis made room by itself
DBSIZE
(integer) 158792              # fewer than 200000 - the rest were evicted
```

Step 3 · See what noeviction does instead

```
CONFIG SET maxmemory-policy noeviction
$ redis-cli DEBUG POPULATE 400000 icici:fill2:
(error) OOM command not allowed when used memory > 'maxmemory'.
```

IMPORTANT noeviction is the default. On a cache it turns a full memory condition into failed writes instead of a slightly smaller cache. Set allkeys-lru on any pure cache, and keep noeviction only where Redis holds data you cannot rebuild.

Step 4 · Snapshots

```
CONFIG SET maxmemory 0
$ redis-cli --scan --pattern 'icici:fill*' | xargs -r redis-cli UNLINK

SET icici:persist:test "before the snapshot"
BGSAVE
Background saving started
```

```

LASTSAVE
(integer) 1758412905
$ ls -lh /var/lib/redis/dump.rdb

$ sudo service redis-server restart
GET icici:persist:test
"before the snapshot"      # survived - it was in the RDB

```

Step 5 · The append-only file

```

CONFIG SET appendonly yes
OK
SET icici:persist:aof "written after AOF was enabled"
$ ls /var/lib/redis/appendonlydir/
appendonly.aof.1.base.rdb  appendonly.aof.1.incr.aof  appendonly.aof.manifest

CONFIG GET appendfsync
1) "appendfsync" 2) "everysec" # at most 1 second of loss
BGREWRITEAOF
Background append only file rewriting started

```

Solution · The persistence decision, written down

Workload	Persistence	Why
Balance / profile cache	None, or RDB only	A restart just means a cold cache - the next miss refills it
Login sessions	AOF everysec	Losing sessions logs every customer out
Rate-limit counters	None	A one-minute window rebuilds itself in a minute
Job queues	AOF everysec	A lost job is work that never happens
Distributed locks	AOF everysec	A lost lock lets two batch runs start

WATCH OUT Restore the defaults before you leave: `CONFIG SET maxmemory 0` and `CONFIG SET maxmemory-policy noeviction`, or restart the service.

CHECKPOINT You have seen eviction reclaim memory, seen noeviction refuse a write, and proved that a key survives a restart.

LAB 7.9 · 45 MINUTES

Cache-aside end to end, in Node.js

Objective	Prerequisite
Put a Redis cache in front of PostgreSQL and measure what it buys you.	Labs 7.1 to 7.8. Node 18+, npm, the PostgreSQL instance from Modules 1 to 6.

Step 1 · Install the client

```
$ mkdir icici-cache && cd icici-cache && npm init -y
$ npm pkg set type=module
$ npm install redis pg
$ node -e "import('redis').then(async ({createClient}) => {" \
    "const c = createClient(); await c.connect();" \
    "console.log(await c.ping()); await c.close(); })"
PONG
```

TIP node-redis command names are camelCase and map one-to-one onto the commands you have been typing: GET is get, HSET is hSet, ZRANGE is zRange. When you cannot find a method, sendCommand(['ANY','COMMAND']) always works.

Step 2 · The cache-aside function

```
import { createClient } from 'redis';
import pg from 'pg';

const r = createClient({ url: 'redis://127.0.0.1:6379' });
r.on('error', (e) => console.error('redis:', e.message));
await r.connect();

const db = new pg.Pool({ database: 'icici', user: 'postgres', password: '****' });

const TTL = 300;
const key = (acct) => `icici:bal:${acct}`;

async function getBalance(acct) {
  const k = key(acct);
  try {
    const cached = await r.get(k);
    if (cached !== null) return [Number(cached), 'HIT'];
  } catch {
    // never fail the request on a cache error
  }

  const { rows } = await db.query(
    'SELECT balance FROM accounts WHERE id = $1', [acct]);
  if (rows.length === 0) return [null, 'MISS'];

  try {
    const jitter = Math.floor(Math.random() * 60);
    await r.set(k, String(rows[0].balance), { EX: TTL + jitter });
  } catch {
    // ignore - the cache is an optimisation, not a dependency
  }
}
```

```

    }
    return [Number(rows[0].balance), 'MISS'];
  }
}

```

Step 3 · Invalidate on write

```

async function postDebit(acct, amount) {
  await db.query(
    'UPDATE accounts SET balance = balance - $1 WHERE id = $2', [amount, acct]);
  await r.unlink(key(acct)); // delete, do not update, and do it here
}

```

TIP Delete rather than write the new value into the cache. A write-through update can race with a concurrent read that is about to populate the cache with the old value. Deleting is always safe - the next read repopulates from the source of truth.

Step 4 · Measure it

```

async function bench(acct, n = 200) {
  await r.unlink(key(acct));

  let t0 = performance.now();
  await getBalance(acct);
  const cold = performance.now() - t0;

  t0 = performance.now();
  for (let i = 0; i < n; i++) await getBalance(acct);
  const warm = (performance.now() - t0) / n;

  console.log(`cold (DB) ${cold.toFixed(2)} ms    ` +
    `warm (cache) ${warm.toFixed(3)} ms    ` +
    `speed-up ${Math.round(cold / warm)}x`);
}

await bench(9003);
cold (DB) 4.81 ms    warm (cache) 0.142 ms    speed-up 34x

```

Step 5 · Check the hit rate

```

$ redis-cli INFO stats | grep -E 'keyspace_(hits|misses)'
keyspace_hits:8412903
keyspace_misses:412088
# hit rate = 8412903 / (8412903 + 412088) = 95.3%

```

Solution · The four rules this lab demonstrates

Rule	In the code	What breaks without it
Always set a TTL	set(k, v, { EX }), never a bare set	A key that is never invalidated is wrong forever
Invalidate at the write	unlink inside postDebit	Stale balances for up to one TTL
Delete, do not update	unlink, not set	A concurrent read can write back the old value
Never fail on a cache error	try/catch around both calls	Redis becoming unavailable takes the whole API down

CHECKPOINT You have a working cache-aside implementation, a measured speed-up, and a hit rate you can quote.

LAB 7.10 · 35 MINUTES

Transactions, Lua and a distributed lock

Objective	Prerequisite
Make multi-step operations atomic, and stop two servers running the same batch job.	Lab 7.9 complete.

Step 1 · MULTI and EXEC

```
SET icici:bal:9003 341244.80
SET icici:cnt:debits 4411

MULTI
  OK
DECRBYFLOAT is not a command - use INCRBYFLOAT with a negative
INCRBYFLOAT icici:bal:9003 -2500
  QUEUED
INCR icici:cnt:debits
  QUEUED
EXEC
  1) "338744.8"
  2) (integer) 4412
```

Step 2 · WATCH - optimistic locking

Open a second redis-cli window for this. Window 1:

```
WATCH icici:bal:9003
  OK
GET icici:bal:9003
  "338744.8"
MULTI
INCRBYFLOAT icici:bal:9003 -1000
  QUEUED
# ---- STOP. Go to window 2 before running EXEC ----
```

Window 2 - change the watched key:

```
SET icici:bal:9003 999999
  OK
```

Back in window 1:

```
EXEC
  (nil) # aborted - the watched key changed
```

WATCH OUT EXEC returning nil is not an error - it is the check-and-set protocol telling you to re-read and try again. Your application code must loop on it.

Step 3 · Why Lua when you already have MULTI

MULTI queues commands before any of them run, so a later command cannot depend on an earlier one's result. Lua can. Rewrite the Lab 7.3 rate limiter as one atomic script:

```
$ cat > /tmp/ratelimit.lua <<'EOF'
```

```

local n = redis.call('INCR', KEYS[1])
if n == 1 then
    redis.call('EXPIRE', KEYS[1], ARGV[1])
end
if n > tonumber(ARGV[2]) then return 0 end
return 1
EOF

$ redis-cli --eval /tmp/ratelimit.lua icici:rl:9003:test , 60 3
(integer) 1          # call 1 - allowed
(integer) 1          # call 2 - allowed
(integer) 1          # call 3 - allowed
(integer) 0          # call 4 - throttled

$ redis-cli TTL icici:rl:9003:test
(integer) 57

```

Step 4 · Cache the script

```

SCRIPT LOAD "return redis.call('INCR', KEYS[1])"
           "9d0c9b3e0f0e1a..."
EVALSHA 9d0c9b3e0f0e1a... 1 icici:cnt:demo
(integer) 1
SCRIPT EXISTS 9d0c9b3e0f0e1a...
1) (integer) 1

```

EVALSHA sends a 40-character hash instead of the whole script body. Clients try EVALSHA first, and fall back to EVAL if the server replies NOSCRIPT after a restart.

Step 5 · A distributed lock

```

# window 1 - acquire
SET icici:lock:eod "token-a1b2c3" NX PX 30000
OK                               # we own it for 30 seconds

# window 2 - another server tries
SET icici:lock:eod "token-x9y8z7" NX PX 30000
(nil)                            # refused - back off and retry
PTTL icici:lock:eod
(integer) 24187                  # how long until it is free

# window 1 - release, but ONLY if we still own it
$ redis-cli --eval - icici:lock:eod , token-a1b2c3 <<'EOF'
if redis.call('GET', KEYS[1]) == ARGV[1] then
    return redis.call('DEL', KEYS[1])
else
    return 0
end
EOF
(integer) 1

```

Solution · Why each part of the lock is there

Part	Removes which failure	What happens without it
NX	Two owners	Both servers run the end-of-day batch

Part	Removes which failure	What happens without it
PX 30000	Deadlock	A crashed holder blocks the job forever
Random token	Deleting someone else's lock	A slow process releases a lock already taken over
Lua on release	A race between GET and DEL	The owner can change between the two commands

IMPORTANT A single-instance lock is lost if that instance fails. For a lock whose loss would be expensive, use Redlock across independent instances - or put the guarantee in PostgreSQL, which is where correctness of record belongs.

CHECKPOINT You can make a batch atomic, detect a concurrent change with WATCH, run atomic logic with Lua, and build a lock that cannot deadlock or be released by the wrong owner.

Cleaning up

Every key created in these labs is prefixed `icici:`. Remove them without touching anything else:

```
$ redis-cli --scan --pattern 'icici:*' | xargs -r redis-cli UNLINK
$ redis-cli DBSIZE
(integer) 0

# restore the defaults changed in lab 7.8
$ redis-cli CONFIG SET maxmemory 0
$ redis-cli CONFIG SET maxmemory-policy noeviction
$ redis-cli CONFIG SET appendonly no
```

What you should be able to do now

- Install and verify Redis on Windows, and connect from `redis-cli`.
- Name keys to a convention, expire them, and find them with `SCAN` instead of `KEYS`.
- Choose between String, Hash, List, Set and Sorted Set from the access pattern.
- Build a rate limiter, a job queue, a segment intersection, a leaderboard and a scheduler.
- Read an encoding, measure a key's memory, and interpret a RESP exchange.
- Configure `maxmemory`, an eviction policy and persistence, and justify each choice.
- Implement cache-aside with a TTL, jitter, invalidation on write and error tolerance.
- Make multi-step operations atomic with `MULTI`, `WATCH` and `Lua`, and build a safe lock.

NEXT Next: the exercise sheet. Attempt it without looking at this guide - then check against the separate solutions document.